Timetabl.ai: Session 1 Summary

Hackathon 1 | Version 1 planning and first build stage
Samiya Sehgal | URN 2302662 | 21 October 2026

Goal: a multi-college (SaaS) web app that builds clash-free timetables for any college and keeps them correct through the semester, with an AI assistant for questions and changes.

1. How I planned before writing code

- Read the brief and decided **not to code** until the product was clear: who uses it, what data it needs, what makes one timetable better than another.
- Worked through about **10 rounds of questions and decisions** with Claude Code, covering roles, rules, algorithm, data input, outputs, chatbot and version scope.
- Reviewed three diagrams (architecture, semester flow, change flow) and sorted every feature into **V1 must / V1 if time / V2** on a board.
- Wrote the decisions into four guide files (root, frontend, backend, chatbot) before building.

2. Key product decisions

- **Roles:** College Admin, Department Coordinator, Teacher, Student. Coordinators manage their own department; teachers change only their own lectures.
- **Changes:** weekly timetable plus one-day changes. A teacher's one-day change is instant if clash-free; a permanent change needs Coordinator approval.
- **Real college cases:** 1 or 2 shifts (strict), sections split by year/department/day scholar, shared rooms borrowed first-come, common classes owned by the admin, lab batches, parallel electives, holidays and semester dates.
- **Lunch:** not fixed. Admin gives the mess window; the algorithm picks each year's lunch period for the best timetable and room use.
- **Students:** simple login with dummy college emails in V1, read-only view and chatbot.

3. Algorithm choice

- **Google OR-Tools CP-SAT** (constraint programming). Hard rules can never be broken, lab blocks are easy to model, and it improves quality within a time limit.
- Rejected greedy (gets stuck), genetic algorithms (can't guarantee zero clashes) and plain integer programming (slower for this problem).
- Edits will use small local re-solves so only affected classes move.
- A separate **clash checker** re-verifies every timetable. It shares no code with the solver.

4. Architecture

Frontend	Next.js, Tailwind, shadcn/ui: all screens and timetable grids
Backend	FastAPI: data, solver, clash checker, Excel/PDF, notifications
Chatbot	Separate service (Claude API), calls backend with the user's own login, so it can't exceed their rights
Data	Supabase Postgres; each college isolated by org_id and row-level security

The algorithm builds timetables; the AI is used only for the chatbot.

5. Version 1 scope (planned)

- College signup and demo college, staff email invites, student login.
- Setup wizard: shifts, lecture and lab length, breaks, holidays, semester dates; Excel import with row errors.
- Coordinator "Ready" flow, then one college-wide generation by the admin with a quality score.
- Section / teacher / room views, publish, Excel and PDF export.
- Drag and drop with live clash check, one-day changes, teacher block times, audit log with undo, notifications.
- Chatbot side panel: answers questions, proposes edits with a before/after view, flags problems after generation.

V2: attendance, teacher booking, WhatsApp/SMS, Google Calendar, NAAC report, Hinglish.

6. What has been done

- Git repository with **3 commits**; project guides for root, frontend, backend and chatbot.
- **Stage 1 (timetable engine) complete:** data models, feasibility check with plain-language errors, CP-SAT generator, independent clash checker, quality score, demo college, command-line runner.
- Handles overlapping shifts sharing rooms and teachers, combined classes in big halls, 2-period labs in parallel batches, teacher blocked times and daily limits, flexible lunch, home-room preference.

Demo college input	12 sections, 37 teachers, 15 venues, 114 allotments, 2 shifts
Result	232 classes placed, **0 clashes**, quality **88/100**, about 60 s
Tests	16 automated tests passing

Problems solved on the way: the first model timed out, so I rewrote it to place each subject allotment as a unit (removing duplicate orderings). Tuning the solver raised quality from 75 to 88. Lunch moved from fixed to solver-chosen after reviewing real college needs.

7. What's next

Stage	Work	Status
1. Engine	Solver, clash checker, quality score, tests	Done
2. See it	API + timetable grid in browser, Generate button	Started
3. Edit it	Drag and drop, local repair, one-day changes, undo	Next
4. Real SaaS	Supabase, login, roles, multiple colleges	To do
5. Set it up	Setup wizard, Excel import, rules page, electives	To do
6. Chatbot	Questions, edit proposals, insights	To do
7. Ship	Publish, exports, notifications, deploy	To do

Immediate next steps: finish the API and timetable grid, push the repository to GitHub, then start drag-and-drop editing.